Why rare classes disappear from a federated generator

The conditioning-row collapse in GeFL-F: what happens, what was already known, what is new, and how to fix it. Plain-English version. All numbers are from our runs (experiments/results).

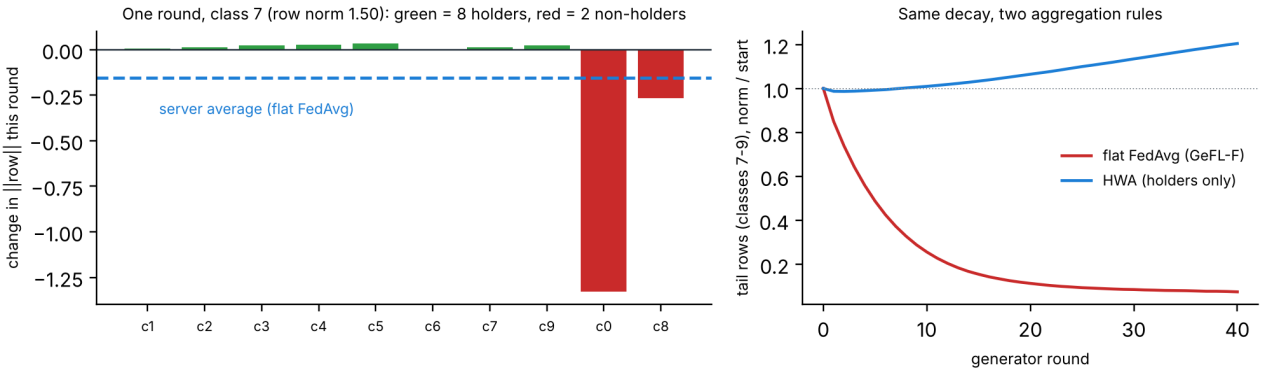
In one paragraph. In GeFL-F every class has its own small vector inside the generator (its *conditioning row*). Only clients that have that class can teach the row anything. Clients that do *not* have the class still touch the row: the optimiser (Adam with weight decay mixed into the gradient) pushes it toward zero, and it pushes hard: a step of the full learning rate, no matter how small the weight decay is. The server then averages every client's copy. For a rare class, the few big pushes toward zero win over the many small useful updates, so the row shrinks every round, the generator forgets the class, and every classifier trained on its samples learns that class badly. This happens even when **most** clients hold the class, not only when fewer than half do, as we first claimed.

1. The setting in plain words

- K clients (here 10). Each has some images, very unbalanced across the 10 classes (the rarest class has 24 training images in total).
- They jointly train one **conditional generator** (a CVAE) that makes fake feature vectors for any class you ask for. The class enters through a **row** of 512 numbers per class. That row is the only part of the generator that belongs to one class.
- Each round: every client copies the global generator, trains it locally for a few hundred steps with **Adam, learning rate 0.001, weight decay 0.001 added to the gradient** (this is called *coupled* weight decay; it is the paper's setting), and sends it back. The server takes the plain average (FedAvg).
- A client that has class c is a **holder** of c; the others are **non-holders**. m_c = number of holders.

2. What happens in one round

A non-holder never sees class c, so the data gives its row zero gradient. The only gradient left is the weight-decay term λ·row. Adam divides every gradient by its own running size, so a tiny but steady gradient still produces a full-size step toward zero. A holder, on the other hand, sees class c only in some batches, and its data gradient fits the row; it does not push the row to grow. The left figure shows a real round of our run: the eight holders changed the row of class 7 by -0.005 to +0.033; the two non-holders changed it by -1.33 and -0.27 (the row's norm was 1.50). The server average moved it by -0.15. Every round repeats this.



Left: per-client change of class 7's row in round 5 (T01, MNIST long tail, K = 10). Right: the tail rows over 40 rounds with flat averaging (red) and with holder-weighted averaging (blue), same optimiser and decay.

3. The result (Theorem 1), stated simply

Measure, over one round, how much each client changes the length of row c. Let **G** be the average gain of the holders and **D** the average loss of the non-holders. Then, after plain averaging:

> new length ≤ old length + $(m_c/K)\cdot G - (1 - m_c/K)\cdot D$
>
> So the row **must shrink** whenever m_c/K is below $\theta_c = D / (G + D)$.

Why it is true. The length of an average is at most the average of the lengths (triangle inequality). Write each client's length as old length plus its change, split the clients into holders and non-holders, done. Three lines; no assumption about the optimiser, the data or the network. It is exact.

Why our first version ("shrinks if fewer than half the clients hold the class") was wrong. That rule is the special case G = D: every holder would have to grow the row exactly as fast as a non-holder shrinks it. In reality holders grow it much more slowly (G is 7 % to 64 % of D in our run), so θ_c is 0.61 to 0.93: the row shrinks even when 7, 8 or 9 of the 10 clients hold the class. ChatGPT's review was right that K/2 did not follow; it actually understated the problem.

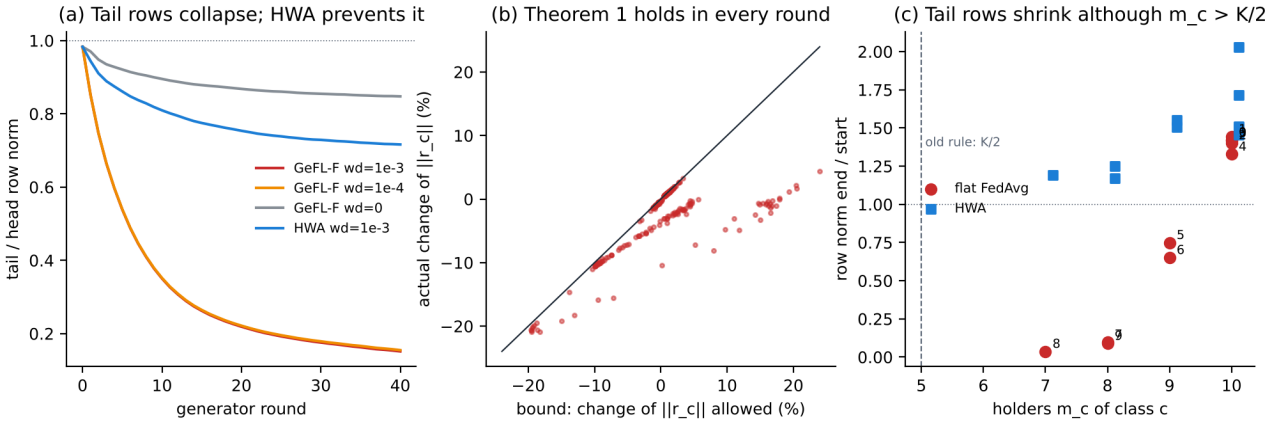
Lemma 2: the size of the weight decay does not matter, only whether it is there. If every gradient a non-holder sees is λ·row, then multiplying λ by 10 multiplies Adam's running mean and its running root-mean-square both by 10, and their ratio, which sets the step, stays the same. So D is the same for λ = 0.0001 and λ = 0.001. With λ = 0 the gradient is exactly zero and the row is untouched (D = 0). With plain SGD the same decay would remove only about steps × 0.001 × 0.001 of the row, roughly 1000 times less.

4. Evidence that it is real

Check (T01: real CVAE-F, MNIST long tail, K = 10, 40 rounds, seed 0)	**Result**
Is the inequality ever violated?	0 violations in 1,200 class-rounds (3 flat-averaging runs)
How many clients hold each tail class?	classes 5-9: 9, 9, 8, 7, 8 of 10 (all above K/2)
Tail rows after 40 rounds, flat FedAvg, λ = 0.001	classes 7, 8, 9: 0.10, 0.04, 0.09 of their start; head rows grow ×1.4
Same with λ = 0.0001 (10× smaller decay)	classes 7, 8, 9: 0.10, 0.04, 0.09; tail/head ratio 0.155 vs 0.152
Same with λ = 0 (no decay)	no collapse: tail/head ratio 0.85; tail rows ×1.20, ×1.14, ×1.12
HWA (average row c over holders only), λ = 0.001	tail rows grow ×1.25, ×1.19, ×1.17 instead of collapsing

Across seeds, datasets and numbers of clients (F01, F04, K03; full schedule): the collapse gets worse as K grows, because each tail class has more non-holders, and HWA prevents it at every K. Both are predictions of Theorem 1.

Tail/head row length at the end (3 seeds)	**K = 10**	**K = 50**	**K = 100**
GeFL-F (flat FedAvg), MNIST	0.33 ± 0.34	0.05 ± 0.03	0.04 ± 0.01
GeFL-F (flat FedAvg), FMNIST	0.22 ± 0.28	0.07 ± 0.02	0.06 ± 0.02
+ HWA + LA, MNIST	0.80 ± 0.09	0.84 ± 0.04	0.86 ± 0.03
+ HWA + LA, FMNIST	1.01 ± 0.15	0.92 ± 0.10	0.93 ± 0.03



(a) tail/head row length over rounds; (b) every class-round: the bound (x) against what happened (y), all on or below the diagonal; (c) final row length against number of holders: classes held by 7-9 of 10 clients collapse under flat averaging (red), not under HWA (blue).

5. What was already known (not ours)

Known result	Source	How it relates
Weight decay added to the gradient behaves differently under Adam than under SGD; fix: decoupled decay (AdamW).	Loshchilov & Hutter, ICLR 2019	Lemma 2 is a direct consequence. We did not discover that Adam changes weight decay.
Rows that get no gradient in a batch should not be updated or decayed ("lazy Adam").	TensorFlow / MindSpore embedding optimisers	Engineering practice for word/item embeddings; the same fix as our LCD. Not analysed, not federated.
When each embedding row is updated by only some clients, aggregate it over the clients that updated it, weighted by their data.	Federated recommendation, e.g. Federated Neural Collaborative Filtering; sparse aggregation (AAAI 2025)	This is essentially HWA, in another domain. We must cite it.
Local training corrupts the classifier weights of classes a client does not have; restrict or calibrate those classes.	FedRS (KDD 2021), FedLC (ICML 2022)	Same family of problem (absent classes), but in the classifier, and without weight decay.
With one class per client, FedAvg can collapse all class embeddings to one point; the server spreads them apart.	FedAwS (Yu et al., ICML 2020)	Same symptom (class vectors collapse), different cause (no negative examples). Must cite.
Averaging over all clients scales a rare class's learning by m_c/K (dilution).	standard FedAvg analysis	Still present after any decay fix; HWA removes it too.

6. What is new (ours)

- That this failure happens **inside a federated conditional generator** (GeFL-F and any generator with per-class parameters trained by FedAvg), and is the reason GeFL-F breaks under long-tailed clients. We found no paper reporting it.
- The **bound with two measurable numbers** (G and D) and the threshold $\theta = D/(G+D)$; the consequence that collapse happens far above half the clients.
- The consequence of Lemma 2 for this setting: tuning the decay coefficient cannot fix it; only removing coupled decay or removing non-holders from the average can.
- A **round-by-round check on the real model** (every client's row logged every round), plus the link to accuracy: GeFL-F's tail recall and generator fidelity fall with the rows.

What is not new and should not be claimed: the triangle inequality itself; that Adam treats coupled decay differently; the idea of averaging a parameter only over the clients that trained it. The contribution is the diagnosis, the measurable condition and the evidence, not new mathematics.

7. Solutions that exist

Theorem 1 says what any fix must do: make D small (stop non-holders from shrinking the row) or take non-holders out of the average. These are the candidates:

Fix	Removes the shrink term D?	Removes the m_c/K dilution?	Where it comes from	Our evidence so far
AdamW (decoupled decay)	yes ($D \approx 0$)	no	Loshchilov & Hutter 2019	not run yet (K15)
No decay on the rows	yes ($D = 0$)	no	common practice for embeddings	MNIST: tail/head 0.83, bal. 69.9 (GeFL-F 68.3); FMNIST: tail/head 0.90, bal. 62.6 (GeFL-F 57.9) — quick pass, 1 seed

Fix	Removes the shrink term D?	Removes the m_c/K dilution?	Where it comes from	Our evidence so far
Lazy decay (decay only rows in the batch)	yes (D = 0)	no	lazy Adam; ours: LCD	MNIST: tail/head 0.79, bal. 69.7 (GeFL-F 68.3); FMNIST: tail/head 0.79, bal. 60.7 (GeFL-F 57.9) — quick pass, 1 seed
HWA: average row c over holders only	yes	yes	ours; same idea as sparse aggregation in fed. recommendation	MNIST: tail/head 0.71, bal. 70.7 (GeFL-F 68.3); FMNIST: tail/head 1.00, bal. 63.4 (GeFL-F 57.9) — quick pass, 1 seed
No class rows at all (class from the exact class mean)	nothing to shrink	nothing to dilute	ours: PC-VAE	PC-VAE inside the final method: see paper §5.11

The quick pass already shows the decay fixes restore the rows (tail/head 0.8-0.9 instead of 0.2-0.6), but it ran with reduced rounds and one seed, and AdamW, the fix a reviewer will ask about first, was never run. That is not enough to say how much HWA adds on top of simply fixing the optimiser.

8. The experiment that settles it (K15, ready to run)

Question. Is HWA better than the standard optimiser fixes, and by how much, in this setting?

- **Arms**: AdamW + LA, lazy decay + LA (new); GeFL-F, + LA, + HWA + LA (already run with the same splits and seeds; reused, not repeated). Heads use LA in every arm, so only the generator differs.
- **Settings**: MNIST and FashionMNIST, 100:1 long tail, Dirichlet(0.5), K = 10, 50 and 100 clients, 3 seeds each: 36 new runs, about 2 GPU-hours per dataset (notebooks NB38, NB39).
- **Measured**: balanced accuracy, tail recall, tail/head row length, generator fidelity on tail classes.

What the theory predicts, written down before the run:

- AdamW and lazy decay bring the tail/head row ratio close to 1 at every K (they set D ≈ 0).
- They still beat + LA (GeFL-F with the bad optimiser) by a large margin.
- HWA is at least as good as both, and its lead **grows with K**, because with more clients a tail class has more non-holders, and only HWA removes their dilution (m_c/K falls for the rarest tail class falls from 0.7 at K = 10 to 0.32 at K = 50 and 0.21-0.24 at K = 100 in these splits).

How we will read it. 18 paired comparisons (2 datasets × 3 K × 3 seeds) of HWA against each fix. If HWA wins at K = 50 and 100 on all seeds (sign test $p < 0.01$) and the gap increases with K, HWA is established as the right fix for this setting and the theory's dilution prediction is confirmed. If AdamW ties HWA everywhere, the honest message changes: the cause is the optimiser, the cheapest fix is AdamW, and the paper's contribution is the diagnosis plus the anchored generator, not HWA. Either outcome is publishable; we will not know which until it runs.

9. Limits of what we have shown

- The round-by-round check (T01) is one dataset, one seed, 40 of the usual 100 rounds. The inequality cannot fail (it is exact); the measured values of G, D and θ could differ on other datasets.
- The bound tells you when a row *must* shrink. A row can also shrink when the bound does not force it, because holders' updates partly cancel in direction (58 of 197 shrinking class-rounds in T01).
- Short row length is the mechanism; what matters for accuracy is generator fidelity and balanced accuracy, which K15 measures directly.